

TD 6 - premiers programmes en promela

Objectif(s)

- ★ Appréhender les spécificités de la programmation avec promela

Exercice(s)

Exercice 1 – Non déterminisme

Considérez le processus ci-dessous :

```
1 proctype proc1(int x)
2 {
3     if
4         :: (x%2==0) -> printf ("x est un nombre pair\n");
5         :: (x == 0) -> printf ("x est nul \n")
6     fi
7 }
```

Question 1

Quels sont les affichages possibles du programme si le processus est activé avec :

- a) `run proc1(2);`
- b) `run proc1(0);`
- c) `run proc1(1);`

Considérez maintenant les processus ci-dessous :

```
1 int x = 2;
2
3 proctype proc_pair( ) {
4     (x%2==0) -> printf ("x:%d est pair\n",x);
5 }
6
7 proctype proc_inc( )
8 {
9     x=x+1;
10    printf ("valeur finale de x=%d\n",x);
11 }
12
13 init{
14     run proc_inc ();
15     run proc_pair();
16 }
```

Question 2

Quels sont les affichages possibles du programme ? L’affichage est-il cohérent ? Si non, comment le corriger ?

Exercice 2 – Boucles

Question 1 – Calcul de la puissance

Ecrivez un programme en promela qui reçoit comme paramètre deux entiers naturels x et y et affiche le nombre x élevé à la puissance y (x^y).

Considérez le processus ci-dessous qui est activé avec deux paramètres : l’entier x , et le booléen boo .

```

1 proctype boucle(short x; bit boo)
2 {
3     do
4         :: (x > 0) ->
5             (boo == true) -> x--;
6
7         :: ( (boo == false) || (x <= 0) ) ->
8             printf ("fin boucle \n");
9             break
10    od;
11 }
```

Question 2

Le processus `boucle` se termine-t-il toujours ? Justifiez votre réponse et proposez une solution, si nécessaire.

Exercice 3 – Échange de messages

On considère deux instanciations du même processus dont l’identifiant, passé au moment de leur activation, est unique ($id=0$ et $id=1$). Le processus 0 attend un caractère depuis l’entrée standard. Celle-ci est représentée par un canal prédéfini `STDIN` sur lequel les données lues sont interprétées comme des entiers. Ce canal doit être déclaré dans le programme, mais il n’est pas nécessaire de préciser son format (puisque celui-ci n’est pas libre).

Dès qu’il a reçu un caractère, le processus 0 l’envoie au processus 1 via un canal. Une fois qu’il a reçu le caractère, le processus 1 l’affiche.

Question 1

Programmez cette synchronisation en promela.

Considérons maintenant N processus organisés en anneau. Le voisin de droite du processus i est le processus $i+1$ sauf pour le processus $N-1$, dont le voisin de droite est le processus 0. Le voisin de gauche du processus i est le processus $i-1$, sauf pour le processus 0, dont le voisin de gauche est le processus $N-1$.

Le processus 0 envoie au processus 1 (voisin de droite) un message contenant son identifiant et une valeur quelconque. Il attend alors recevoir un message de son voisin de gauche, le processus $N-1$, puis affiche la valeur reçue et se termine.

Chacun des autres processus i commence par attendre un message de son voisin de gauche. A la réception de ce message, le processus i incrémente la valeur reçue, l’envoie à son voisin de droite puis se termine.

Question 2

Programmez cette synchronisation en promela.

Exercice 4 – Exclusion mutuelle : l’algorithme de Le Lann

L’algorithme d’exclusion mutuelle de Le Lann est fondé sur l’unicité d’un jeton. Les processus sont organisés en anneau logique. Le jeton circule dans une seule direction par l’envoi de messages. Au début le processus 0 possède le jeton.

Quand un processus reçoit le jeton, il a le droit d’entrer en section critique. En sortant de la section critique ou s’il ne souhaite pas utiliser la section critique, le processus envoie le jeton à son voisin.

Question 1

Programmez en promela l’algorithme d’exclusion mutuelle de Le Lann.

Question 2

Comment pourrait-on vérifier que cet algorithme est correct ?